

Lesson 1: The Address Translation Mechanism and Execution Trace

Slide Range: Slide 1 to Slide 7[cite: 3]

The Big Picture: How Do We Virtualize Fast and Safe? (Slide 1–3)

In the previous chapter, we established that every program runs inside its own private **Virtual Address Space**, isolated from other applications[cite: 3].

Now, consider a major architectural dilemma[cite: 3]:

- Every time a program reads a variable, calls a function, or pushes onto the stack, it uses a **virtual memory address**[cite: 3].
- However, actual physical data resides in real physical RAM chips[cite: 3].
- Who converts that virtual address into a physical address[cite: 3]?

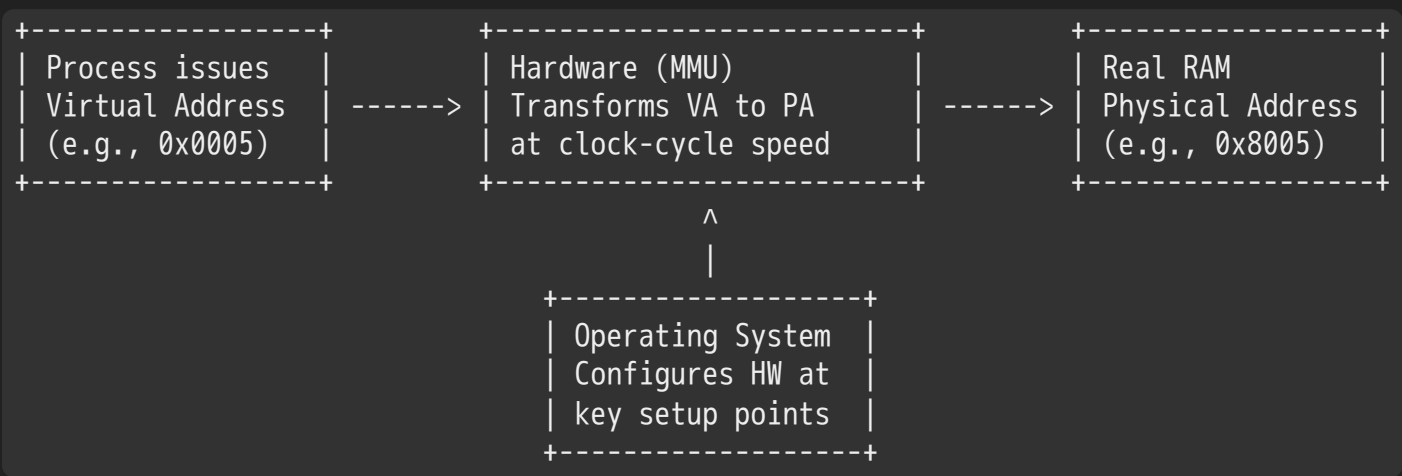
If the Operating System had to convert every single memory reference via software, your computer would crawl:

```
[ Software Trap ] ---> [ Kernel Code ] ---> [ Translation ] ---> [ Return ]
(Takes hundreds of CPU cycles for a SINGLE load or store!)
```

This would make computing unusable.

To solve this, modern systems apply the principle of **Limited Direct Execution (LDE)** to memory virtualization[cite: 3]:

- Programs must run directly on the CPU hardware at native electrical speeds[cite: 3].
- Address translation must happen **in hardware**, not software[cite: 3].
- The operating system steps in only at critical setup and protection junctures[cite: 3].



1. **Hardware Transformation:** The translation from virtual address to physical address is performed on every memory access by hardware—specifically, the **Memory Management Unit (MMU)**[cite: 3].

3].

2. **OS Configuration:** The OS manages memory policies, configures translation registers, and acts as the authority whenever hardware detects a boundary violation[cite: 3].

Walking Through a Memory Reference (Slide 4–6)

To understand what the MMU translates, let us trace a single line of C code down to the instruction level[cite: 3]:

```
c

void func() {
    int x = 3000; // declared on the local stack
    x = x + 3;    // <--- The line we are inspecting
}
```

The compiler translates `x = x + 3;` into three distinct assembly instructions[cite: 3]:

assembly

```
128: movl 0x0(%ebx), %eax    ; 1. Load: Read value from stack memory into %eax
132: addl $0x03, %eax        ; 2. Add: Add 3 to the value inside %eax
135: movl %eax, 0x0(%ebx)    ; 3. Store: Write %eax back out to stack memory
```

- Register `%ebx` holds the virtual memory address of `x` on the stack (located at `15KB` in our example address space)[cite: 3].
- Register `%eax` is a general-purpose CPU scratchpad register[cite: 3].

The Crucial Question: How Many Times Is Memory Accessed? (Slide 6 Annotations)

Students often look at this snippet and guess: *"Two times—once to read `x`, and once to write `x`."*[cite: 3]

That is incorrect[cite: 3]. Memory is accessed 5 separate times[cite: 3]!

Why? Because the CPU cannot execute instructions out of thin air. The assembly instructions themselves live inside physical memory in the **Code Segment**[cite: 3]!

Virtual Address Space Layout:

0KB	+-----+ 128: movl 0x0(%ebx), %eax 132: addl \$0x03, %eax 1KB 135: movl %eax, 0x0(%ebx) +-----+	Program Code Segment
	+-----+ Heap +-----+	
	+-----+ (Free) +-----+	
15KB	[Value: 3000] (Variable x) 16KB +-----+	Stack Segment

Let us trace all 5 operations step-by-step[cite: 3]:

1. **Instruction Fetch (Access 1):** The CPU fetches instruction `movl` from memory address `128` [cite: 3].
2. **Data Read (Access 2):** The CPU executes `movl`, issuing a read request to memory address `15KB` to load `3000` into `%eax` [cite: 3].
3. **Instruction Fetch (Access 3):** The CPU fetches instruction `addl` from memory address `132` [cite: 3].
4. **ALU Operation (Zero Memory Accesses):** The CPU executes `addl`, incrementing `%eax` to `3003` internally within CPU registers[cite: 3].
5. **Instruction Fetch (Access 4):** The CPU fetches instruction `movl` from memory address `135` [cite: 3].
6. **Data Write (Access 5):** The CPU executes `movl`, writing `3003` from `%eax` back into memory address `15KB` [cite: 3].

The Von Neumann Bottleneck (Handwritten Note on Slide 7):

Every instruction executed must first be fetched from memory[cite: 3]. If every memory access required an operating system software routine to translate addresses, running these three simple assembly instructions would take thousands of CPU cycles[cite: 3]! Hardware-level translation via the MMU is necessary to keep system throughput acceptable[cite: 3].

Let me know when you're ready to proceed to **Lesson 2 (Slides 8 to 12)**, where we will examine how hardware implements dynamic relocation using **Base and Bound registers**, address translation arithmetic, and the limits of static chunking[cite: 3]!

2026-09-03 13:09:12

You 

next

Lesson 2: Dynamic Relocation via Base and Bounds, Address Calculations, and Limitations

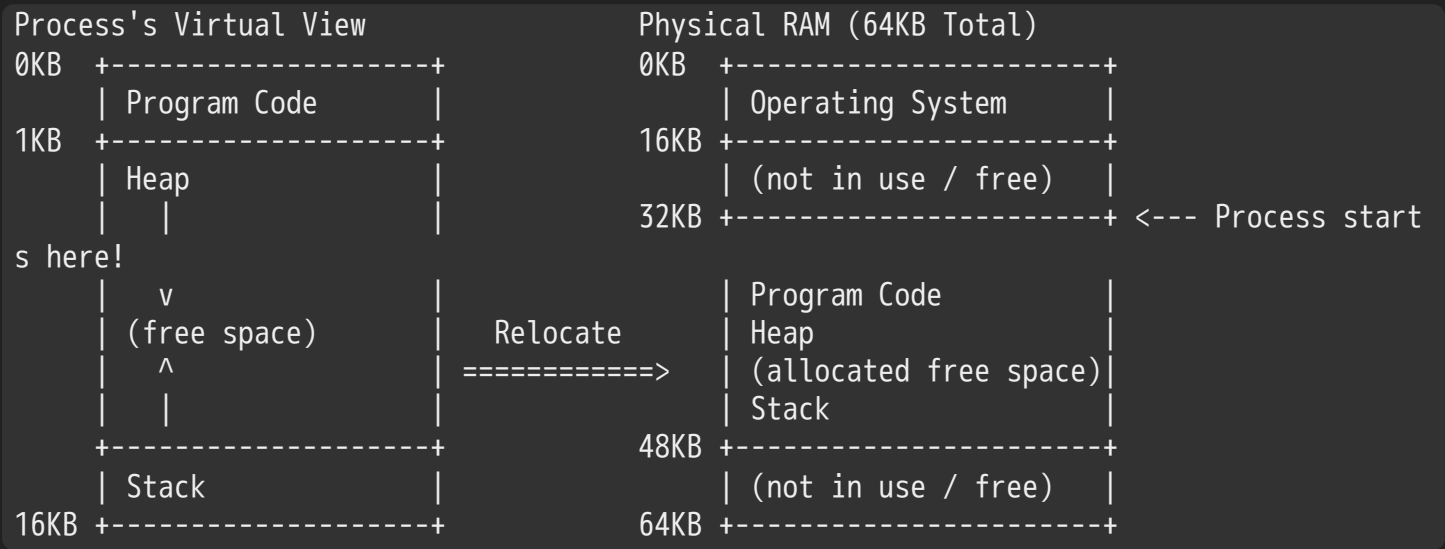
Slide Range: Slide 8 to Slide 12[cite: 3]

The Core Problem: How to Place a Process in Memory (Slide 8)

In Lesson 1, we observed that every program expects its address space to start cleanly at address 0 (or 0x0000 0000) [cite: 3].

Now, consider the physical hardware reality [cite: 3]:

- Physical memory also begins at byte 0 [cite: 3].
- What sits at physical address 0 to 16KB ? The **Operating System itself** [cite: 3]!
- If a program actually tried to place its code at physical address 0 , it would overwrite the kernel and crash the machine [cite: 3].

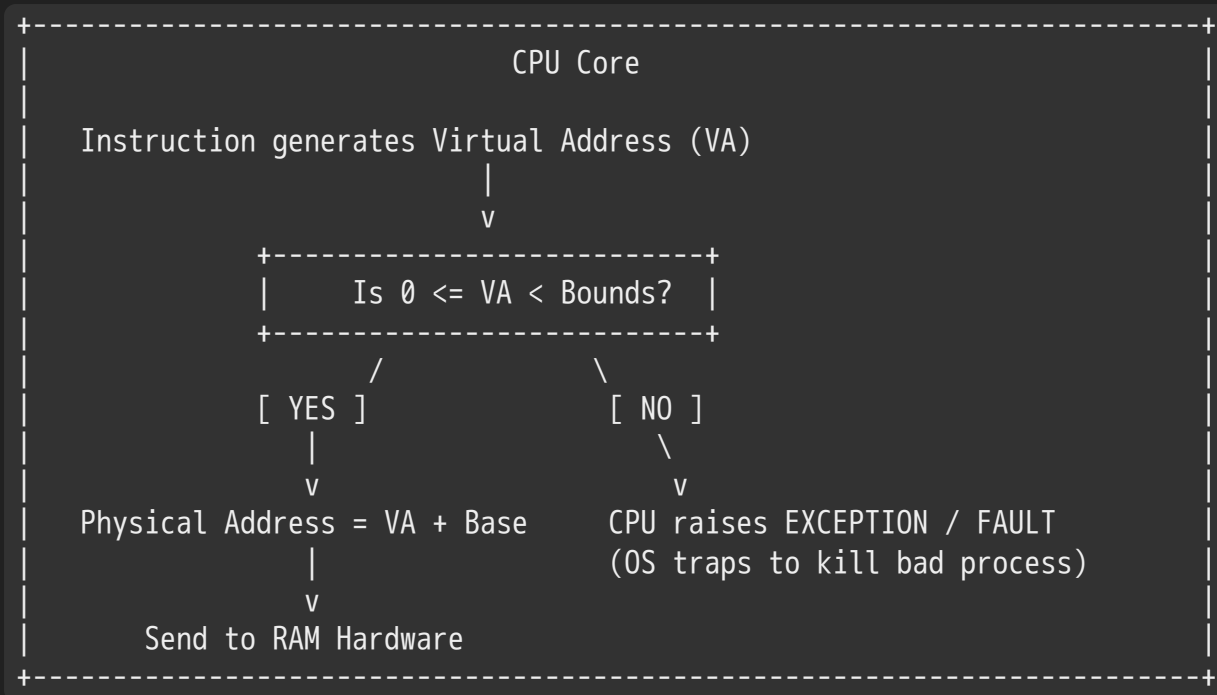


The OS wants the ability to load a process anywhere in physical RAM (for instance, starting at physical byte 32KB), while allowing the process to think it begins at 0KB [cite: 3].

This dynamic mapping is achieved using **Dynamic (Hardware-based) Relocation**, implemented with two dedicated CPU registers: **Base** and **Bounds** [cite: 3].

The Base and Bounds Hardware Mechanism (Slide 9 & 10)

Inside the CPU, two registers are added to the Memory Management Unit (MMU) [cite: 3]:



1. The Base Register (Relocation)

- Holds the physical starting address of the process in RAM[cite: 3].
- In the Slide 9 example, $\text{Base} = 32\text{KB}$ (32768 bytes)[cite: 3].
- Translation formula: $\text{Physical Address} = \text{Virtual Address} + \text{Base}$

2. The Bounds Register (Protection)

- Holds the boundary limit of the address space[cite: 3].
- In the Slide 9 example, $\text{Bounds} = 16\text{KB}$ [cite: 3].
- Validity condition: $0 \leq \text{Virtual Address} < \text{Bounds}$

Why Both Are Necessary (Slide 9 & 10 Notes)

- If the system only used a Base register without Bounds, an attacker inside Process A could generate a virtual address like 18KB [cite: 3].
- Adding that to a base of 32KB would access physical address 50KB —which lies inside another process's space or unallocated memory[cite: 3]!
- The Bounds register ensures no process can reach outside its assigned block[cite: 3].
- If a process references an address $\geq \text{Bounds}$ or generates a negative offset, the MMU halts the operation and triggers an **Illegal Memory Access Exception** (a user-to-kernel trap, resulting in a segmentation fault)[cite: 3].

Concrete Relocation Example (Slide 11)

Let's re-examine our three assembly instructions from Lesson 1, running with $\text{Base} = 32\text{KB}$ (32768 bytes) and $\text{Bounds} = 16\text{KB}$ (16384 bytes)[cite: 3]:

assembly

```
128: movl 0x0(%ebx), %eax    ; %ebx contains virtual address 15KB (15360)
132: addl $0x03, %eax
135: movl %eax, 0x0(%ebx)
```

Instruction 1: Fetch `movl` at Virtual Address 128

1. **Bounds Check:** Is $0 \leq 128 < 16\text{KB}$? Yes[cite: 3].
2. **Translation:** Physical Address = $128 + 32\text{KB} = 128 + 32768 = 32896$ bytes
3. The CPU fetches instruction bytes directly from physical RAM byte `32896` [cite: 3].

Instruction 2: Load Data at Virtual Address 15KB

1. The instruction reads `%ebx`, which holds virtual address `15KB` (15360 bytes)[cite: 3].
2. **Bounds Check:** Is $0 \leq 15360 < 16384$? Yes[cite: 3].
3. **Translation:** Physical Address = $15\text{KB} + 32\text{KB} = 47\text{KB}$ (48128 bytes)
4. The CPU loads the integer `3000` from physical RAM byte `48128` [cite: 3].

The application code executes without awareness of its physical location, while the hardware ensures accesses stay contained[cite: 3].

Two Ways to Define the Bounds Register (Slide 12)

Hardware designers specify the Bounds register using one of two conventions[cite: 3]:

Convention A: Size of Address Space	Convention B: Physical End Address
+-----+	+-----+
Base = 32KB	Base = 32KB
Bounds = 16KB (Size)	Bounds = 48KB (Base + Size)
+-----+	+-----+
Hardware checks:	Hardware computes:
VA < Bounds (16KB)	PA = VA + Base
If legal:	Hardware checks:
PA = VA + Base	PA < Bounds (48KB)
+-----+	+-----+

1. Convention A (Size of Address Space):

- The bounds register stores the total length/size of the allocated address space (e.g., `16KB`) [cite: 3].
- The MMU compares the virtual address against the bounds register first: if $VA < \text{Bounds}$, it adds the base to form the physical address[cite: 3].

2. Convention B (Physical Address of the End):

- The bounds register stores the physical end address in RAM (e.g., $32\text{KB} + 16\text{KB} = 48\text{KB}$)[cite: 3].
- The MMU adds the base to the virtual address first to calculate the candidate physical address, then checks if $\text{PA} < \text{Bounds}$ [cite: 3].

Both provide equivalent protection, though **Convention A** is more common in modern literature because the bounds register remains unchanged even if the process is moved to a different physical base in RAM[cite: 3].

Fatal Limitations of Simple Base and Bounds (Slide 10–11 Annotations)

While simple and fast, pure base-and-bounds has three architectural drawbacks[cite: 3]:

1. Fixed Chunk Size Limits Scalability

- Dividing physical memory into fixed chunks (like 16KB slots across 64KB RAM) restricts the system to very few concurrent processes (e.g., at most 3 or 4 processes)[cite: 3].

2. Process Size is Capped

- If a process needs more than the fixed slot size (e.g., needing 20KB when slots are 16KB), it cannot run at all[cite: 3].

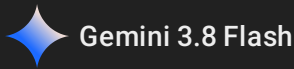
3. Severe Internal Fragmentation

- Notice the diagram in Slide 8: the code and heap occupy the top, and the stack occupies the bottom[cite: 3].
- The large unused region between the heap and the stack must still be allocated in physical RAM because base-and-bounds requires the entire address space to be contiguous[cite: 3].
- This wasted, allocated space inside the block is called **Internal Fragmentation**[cite: 3].

4. Swapping Overhead

- If physical RAM is full and you need to run Process D, you must copy all 16KB of Process A out to the hard disk to free up a slot[cite: 3].
 - Running a process for $20\ \mu\text{s}$ while taking $4\ \text{ms}$ ($4000\ \mu\text{s}$) to swap to disk creates a performance bottleneck[cite: 3].
-

Let me know if you are ready to continue with **Lesson 3 (Slides 13 to 19)**, covering the OS responsibilities during context switching, managing the free list, and the transition toward Segmentation[cite: 3]!



Lesson 3: Hardware Requirements, OS Lifecycle Management, and the Transition to Segmentation

Slide Range: Slide 13 to Slide 19[cite: 3]

The Contract: Hardware Requirements for Dynamic Relocation (Slide 13)

Dynamic relocation cannot work with software alone—it requires strict support from the CPU hardware[cite: 3]. Slide 13 outlines the required hardware features[cite: 3]:

CPU Hardware Support	
1. Dual Mode Execution	Privileged bit: User Mode (restricted) vs. Kernel Mode (privileged).
2. Base & Bounds Registers	One physical pair of registers per CPU core.
3. MMU Arithmetic Circuit	Circuitry to calculate (VA + Base) and check (VA < Bounds) in parallel on every memory clock.
4. Privileged Instructions	Instructions to update Base & Bounds can ONLY be executed by the OS in Kernel Mode.
5. Exception Dispatch Table	Ability to store entry points for OS interrupt and fault handlers.
6. Fault Raising Mechanism	Hardware traps immediately to Kernel Mode if a process tries an illegal or out-of-bounds access.

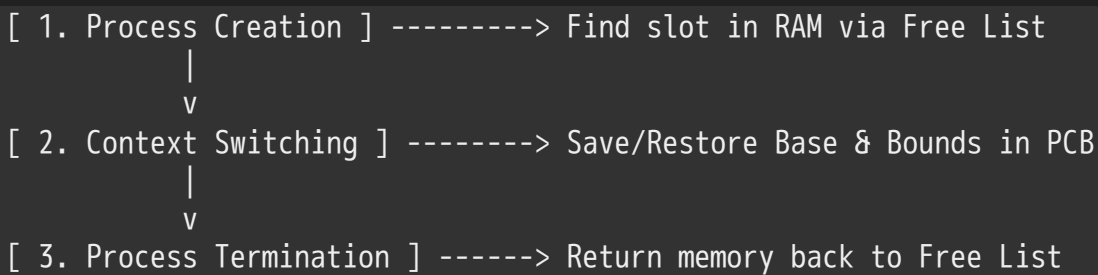
- **Why Must Updating Base/Bounds Be a Privileged Instruction?**[cite: 3]

If a user process could run `movl $0x0, %base_reg`, it could change its own base to 0, point anywhere in memory, overwrite the kernel, or read private data belonging to other processes[cite: 3]. By making this a privileged CPU instruction, any attempt by user code to modify these registers immediately halts execution and triggers a hardware trap into the kernel[cite: 3].

The OS Role: Three Critical Junctures (Slide 14–17)

Since the hardware handles address translation during normal execution, what does the Operating System do[cite: 3]?

The OS intervenes at **three critical junctures** during a program's lifecycle[cite: 3]:



1. When a Process Starts Running: Finding Space (Slide 15)

- When you launch a program, the OS must find a contiguous slot in physical RAM to hold its entire address space[cite: 3].
- To do this, the OS maintains a data structure called the **Free List**—a record of all ranges of physical memory currently unassigned and ready for use[cite: 3].
- In the Slide 15 diagram, RAM is divided into 16KB slots:
 - 0KB – 16KB : Reserved for the OS kernel[cite: 3].
 - 16KB – 32KB : Unused (tracked in Free List)[cite: 3].
 - 32KB – 48KB : In use by an active process[cite: 3].
 - 48KB – 64KB : Unused (tracked in Free List)[cite: 3].
- The OS consults its Free List, picks an unallocated slot (e.g., 16KB–32KB), loads the program code from disk into that slot, and marks that region as used[cite: 3].

2. When Context Switching Occurs (Slide 17)

Suppose Process A is running, its time slice ends, and the OS decides to switch the CPU to Process B[cite: 3].

- **Does the OS copy or move process memory during a context switch?**[cite: 3]
No[cite: 3]! Moving 16KB or megabytes of memory on every context switch would introduce massive overhead[cite: 3]. Physical RAM remains completely unchanged[cite: 3].
- **What changes? Only the CPU registers**[cite: 3]!

Context Switch from Process A to Process B:

1. OS pauses Process A.
2. OS copies current Base & Bounds registers into Process A's PCB:
Process A PCB -> [Base: 32KB | Bounds: 16KB]
3. OS loads Process B's saved Base & Bounds registers from its PCB into the CPU:
CPU Registers <- [Base: 48KB | Bounds: 16KB] (Process B PCB)
4. CPU resumes execution in User Mode at Process B's saved Program Counter.

The hardware MMU now translates every virtual address using Process B's base (48KB) instead of Process A's base (32KB) [cite: 3]. A context switch for memory is simply swapping out register

values[cite: 3]!

3. When a Process Terminates (Slide 16)

- When a program finishes running or is terminated due to an error, the OS does not have to spend time zeroing out all of physical RAM immediately[cite: 3].
- Instead, it takes the physical memory range (e.g., 32KB–48KB) and adds that entry back into its **Free List**[cite: 3]. That block can now be reassigned to the next incoming program[cite: 3].

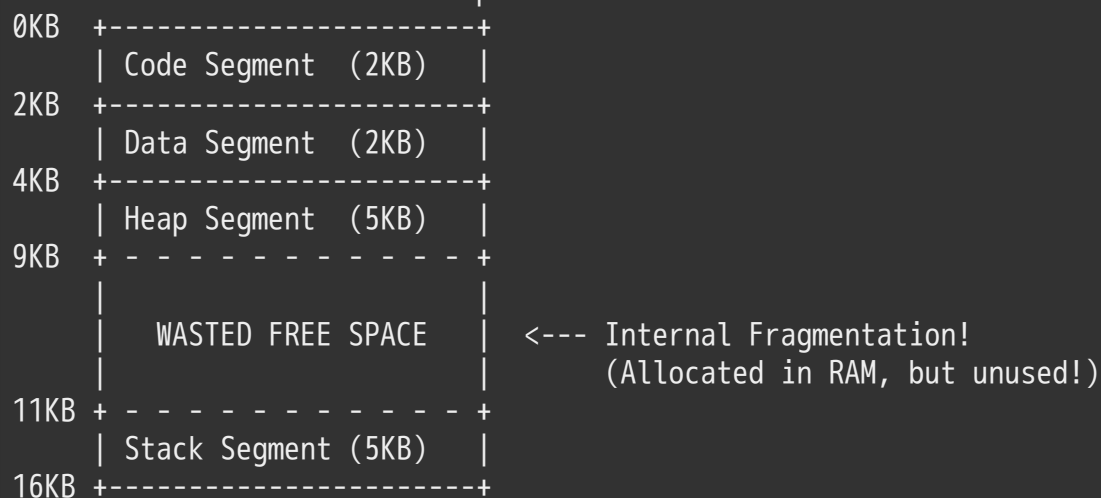
4. Handling Faults & Exceptions (Slide 18)

- At boot time, before any user program is allowed to execute, the OS installs its **Exception Handlers** into the CPU's interrupt vector table using privileged setup instructions[cite: 3].
- If a program generates an illegal virtual address ($\geq$ Bounds or negative), the hardware halts execution and jumps straight to the OS exception handler[cite: 3].
- The handler typically issues a **Segmentation Fault** signal and terminates the offending process to keep the system stable[cite: 3].

The Core Problem of Base and Bounds: Internal Fragmentation (Slide 19)

Slide 19 summarizes this chapter and identifies the fundamental flaw that motivates the next chapter[cite: 3]:

A Process's Virtual Address Space:



- Under simple Base and Bounds, the entire 16KB address space must be placed in physical RAM as a **single, contiguous block**[cite: 3].
- The program only uses 2KB for Code, 2KB for Data, 5KB for Heap, and 5KB for Stack—totaling 14KB[cite: 3].
- The unused gap in the middle must sit inside physical memory doing nothing, wasting RAM[cite: 3]. This is **Internal Fragmentation**[cite: 3].

- Furthermore, if physical RAM does not have a single contiguous 16KB opening, the process cannot run—even if there are plenty of small gaps scattered throughout RAM[cite: 3]!

The Solution: Previewing Segmentation (Slide 19 Annotations)

The handwritten notes on Slide 19 ask a critical design question[cite: 3]:

"How do we reduce the waste in the middle between the heap and the stack? Instead of fixing the size of the whole address space as one block, can we place the segments separately into memory[cite: 3]?"

Instead of ONE Base & Bounds for the whole program: Process Space		Physical RAM
+-----+		+-----+
Code Segment	----->	Code placed at 10KB
+-----+		+-----+
Data Segment	----->	Data placed at 28KB
+-----+		+-----+
Heap Segment	----->	Heap placed at 40KB
+-----+		+-----+
Stack Segment	----->	Stack placed at 60KB
+-----+		+-----+

- **The Core Idea:** Break the address space into logical pieces: **Code**, **Heap**, and **Stack**[cite: 3].
- Give **each segment its own independent Base and Bounds register pair**[cite: 3]!
- Now, segments can be placed wherever free space exists in physical RAM, and the large unused gap in the middle does not have to occupy real memory at all[cite: 3].

This completes all material, diagrams, and annotated concepts for **Chapter 15: Address Translation**[cite: 3].

Whenever you are ready, upload or specify the next chapter (Chapter 16: Segmentation) and we will begin with Lesson 1!